



Chittagong University of Engineering & Technology (CUET)

Department of Computer Science & Engineering (CSE)

Assignment 1

Course Code: CSE-6309

Course Title: Big Data

Submitted by:

Sumitra Das Gupta

ID: 22MCSE101

Submitted to:

Prof. Dr. Mohammad Shamsul Arefin

Professor, CSE, CUET

Date of submission: 12/11/2025

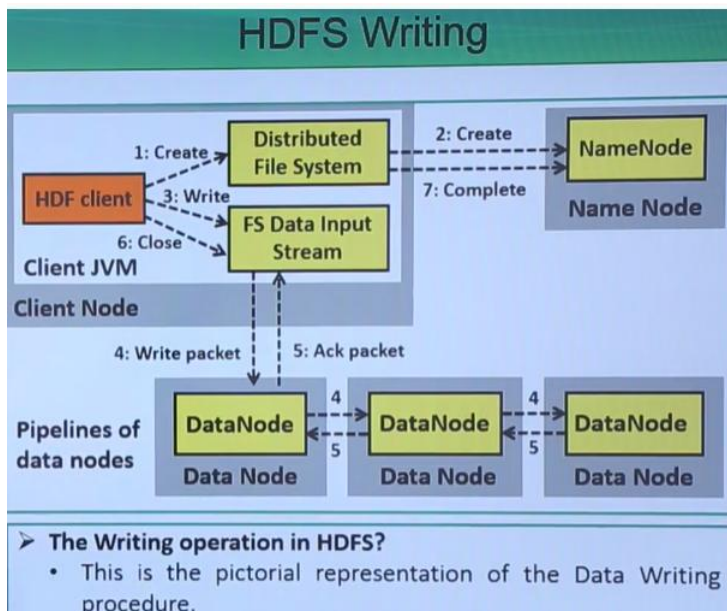
Q1(a)

HDFS splits large files into blocks (default 128MB or 256MB), replicates blocks across DataNodes for reliability, and separates metadata (NameNode) from storage (DataNodes). Clients talk to the NameNode for metadata and to DataNodes for actual block I/O.

Write flow (steps & diagram):

1. Client asks NameNode to create a file (`create` RPC).
2. NameNode checks namespace, allocates block IDs, and returns DataNode pipeline list for the first block (based on rack-awareness and load).
3. Client streams the block to the first DataNode which forwards to the second, etc. — a pipeline write. Each DataNode writes to local disk and pipelines acknowledgments back.
4. After block is filled, client requests next block; process repeats until file end.
5. NameNode updates metadata (block → datanode mapping).

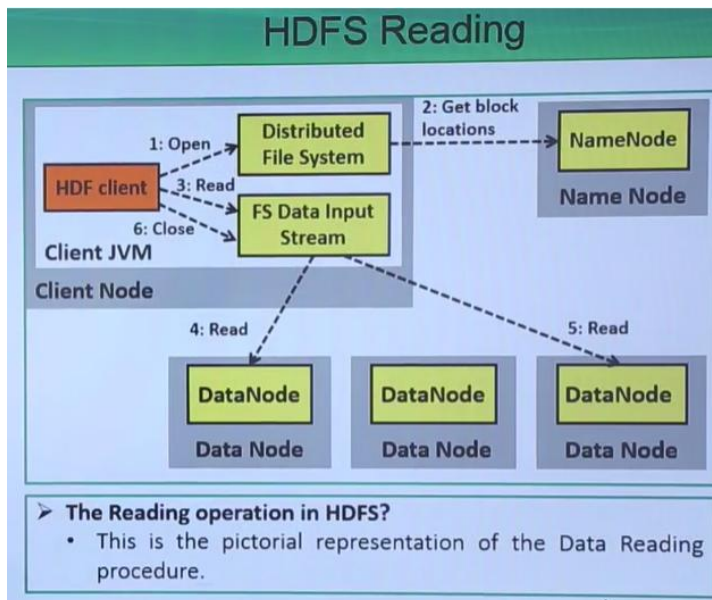
Diagram(write):



Read flow (steps & diagram):

1. Client asks NameNode for block locations for the file.
2. NameNode returns list of DataNodes storing each block (choose local/closest by rack-awareness).
3. Client reads blocks directly from DataNodes (parallel reads possible).
4. If a DataNode is down, client chooses another replica for the block.

Diagram (read):



Q1(b)

- File size = **256 MB**
- Replication factor (RF) = **3**

Total storage used = $256 \text{ MB} \times 3 = 768 \text{ MB}$ (logical HDFS consumption)

If replication factor is increased to 5:

- Storage used** = $256 \text{ MB} \times 5 = 1,280 \text{ MB (1.28 GB)}$.
- Implications:**
 - Storage overhead rises** linearly with RF (more physical disk required).
 - Fault tolerance improves:** can tolerate more simultaneous DataNode failures without data loss (can withstand up to $\text{RF}-1$ replica failures for a block before data is lost).
 - Network & maintenance cost** increases (more replication traffic on writes, more storage to manage).
 - NameNode metadata unaffected** (replicas are metadata entries), but cluster capacity planning must account for increased used space.
 - Read throughput** can improve slightly because clients have more replica choices (better locality), though diminishing returns after a point.

Q1(c)

NameNode (metadata manager)

- Responsibilities: namespace tree, file→block maps, block→DataNode maps, lease management, coordinating replication decisions.
- Interaction: Clients and ResourceManager consult NameNode for block locations; DataNodes send periodic heartbeats & block reports to NameNode.
- Failure impact: If **active NameNode** fails, clients cannot create, delete, or get updated block locations — cluster may become read-only or unavailable for metadata operations unless a High-Availability (HA) setup is present. Single NameNode is a single point of failure (unless HA is configured).

DataNodes (storage workers)

- Responsibilities: store block data, serve reads/writes, send heartbeats and block reports to NameNode.
- Interaction: NameNode instructs DataNodes to replicate or delete blocks. Clients stream directly to DataNodes for IO.
- Failure impact: Data loss risk if replica count falls below RF. NameNode detects missing heartbeats and triggers replication to restore RF. Performance degrades if many DataNodes fail (less parallelism).

Secondary NameNode (checkpointing helper)

- Responsibilities: periodically pulls current fsimage and edit logs from active NameNode, merges them into a new fsimage checkpoint to keep edit log size manageable.
- Interaction: Not a failover NameNode. It simply helps reduce edit-log growth and assists faster NameNode restarts.
- Failure impact: If SecondaryNameNode is down, edit logs on NameNode can grow large — increasing restart time and recovery time — but **does not** directly stop HDFS operation. However absence of checkpointing is risky for long term.

Summary of failure effects

- NameNode down (without HA): cluster metadata unavailable → severe outage (no namespace operations; possible read-only depending on client caches).
- DataNode down: localized data unavailability; NameNode re-replicates blocks; performance drop and temporary under-replication.
- Secondary NameNode down: increased edit log size, longer NameNode restart/recovery times but not immediate data unavailability.

Q2(a)

Big-data approach: In MapReduce, distributed QuickSort is typically implemented as a multi-phase algorithm:

1. **Sampling & pivot selection (Job 0):** mappers sample dataset to choose one or multiple pivots that partition data into ranges (like distributed quickselect / range partitioning).
2. **Partitioning Map stage (Job 1):** each mapper reads elements and emits `<partition_id, value>` where partition is chosen using pivot ranges.
3. **Shuffle:** records for each partition are sent to the reducer responsible for that range.
4. **Reduce stage:** each reducer receives all values in its partition and performs an in-memory or local quicksort (or mergesort). Output reducers' outputs concatenated yields the fully sorted dataset.

Given data: 100, 70, 10, 30, 20, 90, -10, 50, 75, 5, 40, 80, 40, 90, -20

Example distributed steps (illustrative):

1. **Sample & choose pivots:** Suppose sampling yields pivots: 30, 70. This gives three partitions: `(-inf, 30]`, `(30, 70]`, `(70, inf]`.
2. **Map partitioning:** Each mapper reads input list and assigns partition id:
 - Partition1 (≤ 30): 10, 30, 20, -10, 5, 40? (40 goes to next; so not)
 - Partition2 ($30 < \leq 70$): 40, 40, 50, 70, 75? (75 → partition3)
 - Partition3 (> 70): 75, 80, 90, 90, 100
 Exact assignment with our pivots:
 - P1: 10, 30, 20, -10, 5

- P2: 40, 40, 50, 70
 - P3: 75, 80, 90, 90, 100
3. **Reducers sort partitions locally** (each reducer runs quicksort or efficient local sort):
 - Reducer1 sorts P1 → -10, 5, 10, 20, 30
 - Reducer2 sorts P2 → 40, 40, 50, 70
 - Reducer3 sorts P3 → 75, 80, 90, 90, 100
 4. **Concatenate outputs** partition1 + partition2 + partition3 → final sorted list:
 -10, 5, 10, 20, 30, 40, 40, 50, 70, 75, 80, 90, 90, 100

Notes & big-data caveats:

- Pivot selection is crucial to avoid data skew. Choose pivots via random sampling across the dataset.
- If skew occurs (one partition much larger), use more pivots or multi-level partitioning.
- For huge data use external sort on reducer (merge-sort of runs) to handle data larger than reducer memory.

Q2(b)

High-level: YARN separates resource management (cluster level) from application execution. It allows various data processing frameworks (MapReduce, Spark, Flink) to run on a shared cluster.

Components

- **ResourceManager (RM):** global resource manager for the cluster. Responsibilities:
 - Tracks available resources (memory, vcores) across NodeManagers.
 - Manages application queues and schedules containers (via scheduler component).
 - Works with the Scheduler (Capacity, Fair, FIFO) to allocate containers.
- **NodeManager (NM):** per-node agent. Responsibilities:
 - Launch & monitor containers on the node.
 - Report resource availability and container status to RM.
 - Manage local resources (disk, logs).
- **ApplicationMaster (AM):** per-application (job) orchestrator. Responsibilities:
 - Negotiates containers with RM for the application.
 - Coordinates task assignment inside allocated containers.
 - Tracks application progress and handles failures (retries).
- **Containers:** resource bundle (memory + vcores) allocated by RM on a node; the AM launches tasks inside containers (e.g., map tasks, reduce tasks, executors).

How YARN improves scalability vs Hadoop 1.x

- Hadoop 1.x combined resource management and job scheduling in JobTracker (single point, limited scalability). YARN decouples:
 - ResourceManager is lightweight and generic.
 - ApplicationMaster per-job reduces global lock contention and distributes job control logic.
 - Supports multiple frameworks beyond MapReduce (Spark, Tez, Flink).
 - Better resource utilization via fine-grained container allocations and pluggable schedulers.

Q3(a)

- **MPP definition:** A distributed query execution model where a single SQL query is decomposed into many small tasks that execute in parallel across a cluster of worker nodes, each processing a portion of the data and communicating through optimized exchange operators.

- **Impala specifics:**
 - Impala is a low-latency SQL engine for Hadoop that uses MPP: queries are compiled into distributed query fragments executed concurrently on Impala daemons on each node (impalads).
 - Each node scans local HDFS (or HBase) data, applies filters, joins, aggregates, and exchanges intermediate data across nodes using high-speed, RPC-based data exchange (not MapReduce).
 - Benefits: low latency, efficient use of CPU & network, high concurrency for analytic SQL on big data.

Q3(b)

Assuming table `sales_data(sale_id, customer_id, product, category, sale_amount, sale_date, region)`.

i. Sales > 500 in February 2024

```
SELECT *
FROM sales_data
WHERE sale_amount > 500
      AND sale_date >= '2024-02-01'
      AND sale_date < '2024-03-01';
```

ii. Total sales per region

```
SELECT region, SUM(sale_amount) AS total_sales
FROM sales_data
GROUP BY region
ORDER BY total_sales DESC;
```

iii. Top 3 highest sale transactions

```
SELECT sale_id, customer_id, product, category, sale_amount, sale_date, region
FROM sales_data
ORDER BY sale_amount DESC
LIMIT 3;
```

iv. Count of distinct customers per product category

```
SELECT category, COUNT(DISTINCT customer_id) AS distinct_customers
FROM sales_data
GROUP BY category
ORDER BY distinct_customers DESC;
```

v. Average sale amount per category and region

```
SELECT category, region, AVG(sale_amount) AS avg_sale
FROM sales_data
GROUP BY category, region
ORDER BY category, region;
```

(Other tasks like monthly summary or filters for Electronics/North are similar with `DATE_TRUNC` or `EXTRACT`.)

Q3(c)

1. **Client** submits SQL to an **Impala Coordinator** (impalad).
2. **Catalog Service** verifies metadata (table schema, partition info).
3. **Planner/Optimizer** parses SQL, builds logical plan, chooses join order, partition/scan strategies.
4. **Fragmenter** turns plan into distributed fragments and assigns plan fragments to worker impalads.
5. **Execution Engine (MPP)**: worker impalads execute fragments in parallel, scan HDFS/Hive partitions (using Parquet/ORC/other formats), perform local joins/aggregations, and exchange data via network (exchange operators).
6. **Coordinator** collects and merges results, returning final rows to client.

Flow-as-text:

```
Client -> Impala Coordinator -> Catalog -> Planner/Optimizer -> Fragmenter
-> Distributed Impalad Workers (parallel execution & exchange) -> Coordinator -> Client
```

Q4(a)(i)

Big Data definition: datasets whose **volume, velocity, variety, veracity**, and/or **value** exceed the capabilities of traditional systems to capture, store, manage, and process efficiently. Big Data systems are designed to handle petabytes, high ingestion rates, heterogeneous formats, and analytics at scale.

Structured vs Unstructured

- **Structured data:** organized with fixed schema (tables, rows, columns). Example: relational database records.
- **Unstructured data:** no fixed schema — text, images, audio, logs, JSON, social media posts.

Three key RDBMS limitations for Big Data

1. **Scalability (vertical vs horizontal):** Traditional RDBMS scale vertically (bigger server) — expensive and limited. Big Data demands horizontal scaling across commodity nodes.
2. **Schema rigidity:** RDBMS require predefined schema; ingesting variable/unstructured data (JSON, logs) is cumbersome.
3. **Throughput for analytics:** OLAP on very large datasets (PBs) with many concurrent analytical queries is slow or cost-prohibitive on RDBMS; distributed file systems and MPP engines deliver better throughput.

Q4(a)(ii)

Example: Social media platform (high write volume and flexible user content structures). NoSQL (e.g., Cassandra or HBase) is preferable because:

- It supports high write throughput and linear scaling across nodes.
- It accepts semi-structured data without strict schema changes.
- Provides tunable consistency and availability suitable for large distributed user bases.

Q4(a)(iii)

Impact on daily life & society

- Personalized services (recommendation engines), improved healthcare analytics, traffic optimization, fraud detection, targeted advertising, real-time monitoring in smart cities.
- Enables data-driven decision making across government, business, and research.

Two challenges implementing Big Data at scale

1. **Data governance & privacy:** Ensuring data quality, lineage, and compliance (GDPR, etc.) across massive distributed stores.
2. **Operational complexity & cost:** Building and maintaining scalable pipelines, clusters, and ML model deployment requires skilled teams and significant infrastructure/operational tooling.

Q4(b)(i)

Effectiveness

- Provides high-resolution aerial imagery for crop health (NDVI), water stress detection, pest/disease spotting, and precise spraying — enabling precision agriculture.
- When combined with ML/time series analytics, yields predictions for yield, fertilization needs, and targeted interventions — saving cost and increasing yield.

Limitations farmers might face

- **Costs & skills:** Drones, sensors, and analytics pipelines require capital and trained operators/analysts.
- **Connectivity & data handling:** Rural areas may lack bandwidth; transferring and processing large imagery datasets is challenging.
- **Regulation & safety:** Local aviation rules and privacy concerns can limit use.
- **Data interpretation:** Accurate models need local calibration; generic models can give misleading advice.

Q4(b)(ii)

Requirements: very high ingestion (100M/day $\approx$ ~1,157 posts/sec on average, peak bursts higher), near-real-time sentiment processing, storage for historical trends, scalable analytics and model serving.

Recommended Big Data stack (components + why):

1. **Ingestion: Apache Kafka** (durable, high-throughput, partitions for parallelism) to buffer and stream posts.
2. **Stream Processing: Apache Flink** or **Spark Structured Streaming** for real-time processing and exactly-once semantics; Flink preferred for lower-latency and stateful streaming (windowed aggregations for trends).
3. **Preprocessing & Enrichment:** Use lightweight microservices (Docker) for language detection, normalization; or serverless functions for heavy tasks.
4. **Model inference:** Serve sentiment models with **TensorFlow Serving**/TorchServe or use optimized on-the-fly models in Flink using ONNX for lower latency; use GPUs for batch model training; inference can run on CPU depending on model size.
5. **Storage:**
 - **Hot store** for recent trends: **Elasticsearch** (fast text queries, aggregations, dashboards).
 - **Long-term store:** **HDFS**, **S3**, or cloud object store for raw posts and historical analytics.
 - **Key-value store** for state: **RocksDB** (Flink state backend) or **Cassandra/HBase** for lookup access.
6. **Batch analytics / ML training:** **Spark on YARN** or Databricks for large-scale model training and feature engineering.
7. **Monitoring & Observability:** Prometheus + Grafana, Kafka Manager, and alerting for pipeline health.

8. **Orchestration & Scaling:** Kubernetes for microservices; autoscaling for stream consumers; partitioning in Kafka to scale consumers.

Why these choices: Kafka + Flink gives robust, scalable, low-latency pipeline; Elasticsearch supports fast queries and dashboards; object store + Spark supports heavy offline training.

Q5

Cluster summary (given): 4 masters, 20 DataNodes (64 GB RAM, 16 vCores, 10 TB each), RF=3, Capacity Scheduler, Hadoop 3.3.6.

Part (a) HDFS Troubleshooting

1) Possible reasons for under-replicated files & missing blocks

- DataNode failures (hardware/network) causing replicas lost.
- Disk failures or disks full (blocks cannot be replicated).
- Network partitions preventing replication pipeline.
- Admin operations (manual deletions or misconfigured rack awareness).
- NameNode corruption or delayed block reports preventing correct counts.
- Sudden high load preventing replication threads from catching up.

2) Step-by-step process + commands to identify & repair

Identify under-replicated & corrupt blocks

1. Check cluster health & report

```
hdfs dfsadmin -report
```

2. List under-replicated blocks

```
hdfs fsck / -includeSnapshots -blocks -locations | grep -A3 "Under replicated"
# Or use:
hdfs fsck / -blocks -locations | egrep "Under replicated block|MISSING"
```

3. Show corrupt files

```
hdfs fsck / -list-corruptfileblocks
```

4. List DataNode status (heartbeats)

```
hdfs dfsadmin -report
# check for dead nodes section
```

Repair procedure

1. **If missing replicas due to dead DataNodes:** bring DataNodes back online (fix OS/hardware/network), restart DataNode service:

```
sudo systemctl restart hadoop-hdfs-datanode
```

2. Trigger replication manually (set replication or re-replicate a file)

```
# Re-replicate file to desired factor (wait -w for completion)
hdfs dfs -setrep -w 3 /path/to/file
```

3. If blocks are corrupt, remove corrupted files or restore from backup

```
# Move corrupted file to safe location
hdfs dfs -rm /path/to/corruptfile
# Or restore from backup/Snapshots
```

4. Run balancer to even out storage and replicas after repair:

```
hdfs balancer -threshold 10
# threshold = % deviation allowed
```

5. Force NameNode re-check and safe mode handling

```
# Leave safe mode (if stuck)
hdfs dfsadmin -safemode leave
# Enter safe mode for maintenance
hdfs dfsadmin -safemode enter
```

3) Block size selection effect on performance

- **Too small block size** → many blocks → NameNode metadata explosion, higher seek/IO overhead and slow job startup (small file problem).
- **Too large block size** → fewer parallel tasks (fewer map tasks) potentially under-utilizing cluster CPU for highly parallel workloads.
- Choose block size matched to workload: for huge files & sequential processing, larger blocks (128–256 MB). For many small objects, use container formats (SequenceFile, Avro, Parquet) or HDFS archival (HAR) and enable HDFS federation/NameNode HA.

Part (b) MapReduce performance bottlenecks

Major causes (≥3)

1. **Mapper/reducer imbalance (data skew):** one reducer receives much more data causing stragglers.
2. **Small files problem:** too many tiny files cause many short map tasks and high NameNode pressure.
3. **I/O bottlenecks:** slow disk, network congestion or poor data locality increase job time.
4. **Misconfigured memory & sorting parameters:** spills to disk frequently due to small `io.sort.mb` causing excessive disk I/O.
5. **Excessive GC or too small heap for containers** leading to JVM pauses.

Tuning parameters & suggestions

- **io.sort.mb** (default ~100 MB): increase to reduce spills (e.g., `mapreduce.task.io.sort.mb=256` if node memory allows).
- **mapreduce.task.io.sort.factor** (merge factor): increase to reduce number of merge passes (example: `mapreduce.task.io.sort.factor=100`).
- **mapreduce.map.memory.mb / mapreduce.reduce.memory.mb:** allocate according to node memory (e.g., for 64GB node with 16 vCores, you may allow ~3–4 GB per container depending on concurrency).
- **mapreduce.map.java.opts / mapreduce.reduce.java.opts:** set appropriate JVM heap (e.g., `-Xmx` ~75–80% of container memory).

- **mapreduce.job.reduces:** set reducers based on data size and cluster capacity (not too few causing long reduce phases; too many causes overhead).
- **Combine small files:** use `CombineFileInputFormat` or merge files into larger containers (`SequenceFile`, `Avro`, `Parquet`).
- **Speculative execution:** turn on (`mapreduce.job.speculative=true`) to mitigate stragglers or tune thresholds.
- **Compression:** enable intermediate compression (`mapreduce.map.output.compress=true`) to reduce shuffle I/O.

Profiling using JobHistory & Counters

- **Open JobHistory server UI** (`HistoryServer`) and inspect job details:
 - Task-level timeline, slowest tasks, task attempts.
 - Counters: `HDFS_BYTES_READ`, `REDUCE_SHUFFLE_BYTES`, `MAP_OUTPUT_RECORDS`, spill counts.
- **Identify hotspots**
 - Large shuffle size or many spills → tune sort and memory.
 - One reducer with high input → data skew, consider partitioner or custom partition function.
- **Use counters to estimate**
 - `MAP_INPUT_RECORDS` VS `MAP_OUTPUT_RECORDS` indicates map selectivity.
 - `REDUCE_INPUT_GROUPS` tells number of keys per reducer.
- **Example commands**

```
# Get job counters
yarn logs -applicationId <appId> | grep -i counters -A 20
# Or use CLI to view job history JSON
hadoop job -history <jobhistoryfile>
```

Part (c) YARN resource mismanagement

How Capacity Scheduler allocates resources & causes of imbalance

- Capacity Scheduler assigns a percentage of cluster resources to named queues. Each queue can have multiple users and subqueues; resources are divided per configured capacity.
- **Uneven utilization causes**
 - Static capacities not tuned to workload (some queues configured too small or too large).
 - Users/jobs with high concurrency can monopolize queue capacity if ACLs/limits not set.
 - Misconfigured maximum capacities or lack of preemption allows some queues to starve.
 - Misplacement of workloads (e.g., latency workloads in analytic queue).

Corrective actions

- **Adjust queue capacities** to reflect real workload patterns (increase capacity for bursty queues).
- **Enable resource preemption** to reclaim capacity from idle/over-consuming queues and satisfy guaranteed capacities for others.
- **Configure user limits and maximum applications per queue** to avoid single user monopolization.
- **Move to Fair Scheduler** if workloads require dynamic fair share between many jobs (especially for mixed interactive + batch).
- **Implement queue partitioning:** dedicated low-latency queue for streaming/real-time and batch queue for large jobs.

in yarn-site.xml

```
<!-- Use Capacity Scheduler -->
<property>
  <name>yarn.resourcemanager.scheduler.class</name>

  <value>org.apache.hadoop.yarn.server.resourcemanager.scheduler.capacity.CapacityScheduler</value>
</property>

<!-- Enable preemption -->
<property>
  <name>yarn.resourcemanager.scheduler.monitor.enable</name>
  <value>true</value>
</property>
<property>
  <name>yarn.resourcemanager.monitor.policy</name>

  <value>org.apache.hadoop.yarn.server.resourcemanager.monitor.capacity.ProportionalCapacityPreemptionPolicy</value>
</property>
```

capacity-scheduler.xml (example short excerpt)

```
<allocations>
  <queuePlacementPolicy>leafQueue</queuePlacementPolicy>
  <queues>
    <queue>
      <queueName>root</queueName>
      <queues>
        <queue><queueName>realtime</queueName></queue>
        <queue><queueName>analytics</queueName></queue>
        <queue><queueName>adhoc</queueName></queue>
      </queues>
    </queue>
  </queues>

  <queue>
    <queueName>root.realtime</queueName>
    <capacity>20.0</capacity>          <!-- 20% capacity -->
    <maximum-capacity>40.0</maximum-capacity>
  </queue>
  <queue>
    <queueName>root.analytics</queueName>
    <capacity>60.0</capacity>          <!-- 60% capacity -->
    <maximum-capacity>80.0</maximum-capacity>
  </queue>
  <queue>
    <queueName>root.adhoc</queueName>
    <capacity>20.0</capacity>          <!-- 20% capacity -->
  </queue>
</allocations>
```

Part (d) Scheduling techniques and optimization

Which scheduler for mixed real-time + batch workload?

- **Best choice: Capacity Scheduler** with dedicated queues OR **Fair Scheduler** with queue preemption depending on requirements.
 - If the research center already uses **Capacity Scheduler**, keep it but configure a **realtime** queue with guaranteed capacity and enable preemption so latency-sensitive jobs get prompt containers.
 - **Fair Scheduler** is also a good option for mixed workloads because it dynamically shares resources and can give quick resources to interactive jobs when idle capacity exists.

Why: You need predictable capacity for streaming/real-time workloads (latency sensitivity) and high throughput for batch analytics. Capacity scheduler offers predictable capacity guarantees; Fair provides dynamic sharing — choose based on SLA strictness.

Incorporating job priorities, deadlines, SLAs

- Create separate queues: `realtime` (guaranteed min capacity, high priority), `batch` (large capacity, lower priority).
- Use **YARN reservations** or application priorities (via `yarn.scheduler.capacity.<queue>.priority` or Fair Scheduler weights).
- Use **preemption policy** (proportional preemption) to pull resources from lower-priority queues if SLA-guaranteed queue needs them.
- Use **application-level deadline enforcement**: AM can request container resources with higher priority; integrate with scheduling policies or external workflow manager (e.g., Oozie/Airflow) to enforce deadlines.

Advanced schedulers

- **Delay Scheduling**: helps locality by briefly delaying assignment of tasks to improve data locality and reduce network traffic.
- **Dominant Resource Fairness (DRF)**: used in multi-resource contexts (CPU & memory) to ensure fairness across multiple resource types and to avoid starvation.

Part (e) System-wide diagnosis & reporting

Weekly diagnostic checklist & workflow (proactive)

1. **Disk & HDFS utilization**
 - Metric: % used per DataNode, free space per disk.
 - Command: `hdfs dfsadmin -report` and `hdfs fsck / -blocks -files`.
2. **Node health & resource usage**
 - Metric: CPU load, memory usage, io wait, per-node container count.
 - Check NodeManager UI or `yarn node -list` and OS metrics (`top`, `iostat`).
3. **Network I/O & latency**
 - Metric: network throughput and packet errors between nodes (esp. racks).
 - Use `ifstat`, `sar`, or monitoring tool network graphs.
4. **HDFS integrity**
 - Metric: number of under-replicated blocks, corrupt blocks.
 - Commands: `hdfs fsck / -list-corruptfileblocks`, `hdfs fsck / -blocks -locations`.
5. **YARN resource reports**
 - Metric: queue utilization, active vs pending containers, queue-wise latency.

- Check ResourceManager UI and Capacity Scheduler metrics.

Three recommended open-source tools

- **Prometheus + Grafana** — metrics collection and dashboarding for nodes, HDFS, YARN, Kafka, etc.
- **Apache Ambari** (or Cloudera Manager if available) — cluster management, alerts, easier HDFS/YARN ops.
- **Elasticsearch + Kibana / Logstash (EFK)** — centralized logs and quick search to debug errors; or use **Graylog**.

Automation recommendation

- Write small shell scripts or cron jobs to run key checks and post results:
 - Example cron job (run weekly):
 - `#!/bin/bash`
 - `# weekly_check.sh`
 - `hdfs dfsadmin -report > /var/log/bigdata/hdfs_report_$(date +%F).log`
 - `hdfs fsck / -list-corruptfileblocks > /var/log/bigdata/corrupt_$(date +%F).log`
 - `yarn node -list > /var/log/bigdata/yarn_nodes_$(date +%F).log`
 - `# send email or push to Slack if thresholds exceeded`
- Integrate Prometheus exporters (Node Exporter, JMX Exporter for Hadoop) and create Grafana alerts for thresholds (under-replicated blocks > 0, disk > 85%, queue pending containers > X).